

# Lab 4: The Numerical Solver

EE0849 Introduction to Robotics

Basri Erdogan

## Table of contents

|          |                                                                    |           |
|----------|--------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Lab Overview</b>                                                | <b>1</b>  |
| 1.1      | Objectives . . . . .                                               | 2         |
| 1.2      | Prerequisites . . . . .                                            | 2         |
| 1.3      | Deliverables . . . . .                                             | 2         |
| <b>2</b> | <b>Part 1: Setup and Review (10 minutes)</b>                       | <b>3</b>  |
| 2.1      | 1.1 Create Lab File . . . . .                                      | 3         |
| 2.2      | 1.2 Exercise 1: Verify Jacobian . . . . .                          | 3         |
| <b>3</b> | <b>Part 2: Basic Newton-Raphson Solver (25 minutes)</b>            | <b>5</b>  |
| 3.1      | 2.1 Implement the Solver . . . . .                                 | 5         |
| 3.2      | 2.2 Exercise 2: Test the Solver . . . . .                          | 5         |
| 3.3      | 2.3 Visualize Convergence . . . . .                                | 6         |
| <b>4</b> | <b>Part 3: The Singularity Problem (20 minutes)</b>                | <b>7</b>  |
| 4.1      | 3.1 Test Near Singularity . . . . .                                | 7         |
| 4.2      | 3.2 Exercise 3: Observe Singularity Behavior . . . . .             | 7         |
| <b>5</b> | <b>Part 4: Damped Least Squares (25 minutes)</b>                   | <b>8</b>  |
| 5.1      | 4.1 Implement DLS Solver . . . . .                                 | 8         |
| 5.2      | 4.2 Exercise 4: Compare Methods . . . . .                          | 8         |
| 5.3      | 4.3 Exercise 5: Damping Parameter Study . . . . .                  | 8         |
| <b>6</b> | <b>Part 5: Initial Guess Effects (15 minutes)</b>                  | <b>10</b> |
| 6.1      | 5.1 Exercise 6: Different Solutions . . . . .                      | 10        |
| <b>7</b> | <b>Part 6: Using roboticstoolbox (15 minutes)</b>                  | <b>11</b> |
| 7.1      | 6.1 Compare with Library . . . . .                                 | 11        |
| 7.2      | 6.2 Exercise 7: FANUC IK . . . . .                                 | 11        |
| <b>8</b> | <b>Part 7: Challenge - Handle Unreachable Targets (10 minutes)</b> | <b>12</b> |
| 8.1      | 7.1 Implement Safe IK . . . . .                                    | 12        |
| <b>9</b> | <b>Summary</b>                                                     | <b>13</b> |
| 9.1      | Checklist . . . . .                                                | 13        |
| 9.2      | Key Takeaways . . . . .                                            | 13        |
| 9.3      | Next Week . . . . .                                                | 13        |

## 1 Lab Overview

## 1.1 Objectives

In this lab, you will:

1. Implement a numerical IK solver using the Jacobian
2. Watch the virtual robot “slide” toward the target iteratively
3. Handle singularities using Damped Least Squares
4. Explore the effect of initial guesses on solutions
5. Gracefully handle unreachable targets

## 1.2 Prerequisites

- Completed Lab 3 (analytical IK)
- Understanding of the Jacobian as a “direction guide”
- Basic linear algebra (matrix inverse)

## 1.3 Deliverables

- Working numerical IK solver
- Visualization of iterative convergence
- Comparison with analytical solutions
- Error handling for edge cases

## 2 Part 1: Setup and Review (10 minutes)

### 2.1 1.1 Create Lab File

Create lab4\_numerical\_ik.py:

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation

# Link parameters
L1 = 1.0
L2 = 0.5

def fk_2link(q):
    """Forward kinematics."""
    theta1, theta2 = q
    x = L1*np.cos(theta1) + L2*np.cos(theta1 + theta2)
    y = L1*np.sin(theta1) + L2*np.sin(theta1 + theta2)
    return np.array([x, y])

def jacobian_2link(q):
    """Compute Jacobian."""
    theta1, theta2 = q
    s1, c1 = np.sin(theta1), np.cos(theta1)
    s12 = np.sin(theta1 + theta2)
    c12 = np.cos(theta1 + theta2)
    return np.array([
        [-L1*s1 - L2*s12, -L2*s12],
        [L1*c1 + L2*c12, L2*c12]
    ])

def plot_arm(ax, q, color='blue', alpha=1.0):
    """Plot the 2-link arm."""
    theta1, theta2 = q
    x1 = L1*np.cos(theta1)
    y1 = L1*np.sin(theta1)
    x2, y2 = fk_2link(q)
    ax.plot([0, x1, x2], [0, y1, y2], 'o-', color=color,
            linewidth=3, markersize=8, alpha=alpha)
    return x2, y2
```

### 2.2 1.2 Exercise 1: Verify Jacobian

Test that your Jacobian is correct using finite differences:

```
def jacobian_numerical(q, epsilon=1e-6):
    """Compute Jacobian numerically."""
    J = np.zeros((2, 2))
    for i in range(2):
        q_plus = q.copy()
        q_minus = q.copy()
        q_plus[i] += epsilon
        q_minus[i] -= epsilon
        J[:, i] = (fk_2link(q_plus) - fk_2link(q_minus)) / (2*epsilon)
```

```
    return J

# Test
q_test = np.array([0.5, 0.3])
J_analytical = jacobian_2link(q_test)
J_numerical = jacobian_numerical(q_test)
print("Analytical:\n", J_analytical)
print("Numerical:\n", J_numerical)
print("Max difference:", np.max(np.abs(J_analytical - J_numerical)))
```

**Record:** Max difference = \_\_\_\_\_ (should be  $< 1e-8$ )

## 3 Part 2: Basic Newton-Raphson Solver (25 minutes)

### 3.1 2.1 Implement the Solver

```
def numerical_ik(target, q0, tol=1e-6, max_iter=100):
    """
    Numerical IK using Newton-Raphson.

    Parameters:
        target: [x, y] target position
        q0: Initial guess [theta1, theta2]
        tol: Convergence tolerance
        max_iter: Maximum iterations

    Returns:
        q: Solution
        history: List of intermediate q values
        success: Whether converged
    """
    q = np.array(q0, dtype=float)
    history = [q.copy()]

    for i in range(max_iter):
        # Current position
        pos = fk_2link(q)

        # Error
        error = np.array(target) - pos

        # Check convergence
        if np.linalg.norm(error) < tol:
            return q, history, True

        # Jacobian
        J = jacobian_2link(q)

        # Update:  $q_{new} = q + J^{-1} * error$ 
        dq = np.linalg.solve(J, error)
        q = q + dq
        history.append(q.copy())

    return q, history, False
```

### 3.2 2.2 Exercise 2: Test the Solver

```
# Target
target = [1.0, 0.6]

# Initial guess
q0 = [0.2, 0.3]

# Solve
q_sol, history, success = numerical_ik(target, q0)
```

```

print(f"Target: {target}")
print(f"Solution: 1={np.degrees(q_sol[0]):.2f}°, 2={np.degrees(q_sol[1]):.2f}°")
print(f"Iterations: {len(history)}")
print(f"Success: {success}")

# Verify
pos_final = fk_2link(q_sol)
print(f"Final position: ({pos_final[0]:.6f}, {pos_final[1]:.6f})")
print(f"Error: {np.linalg.norm(pos_final - target):.2e}")

```

Record your results:

- Iterations: \_\_\_\_\_
- Final error: \_\_\_\_\_

### 3.3 2.3 Visualize Convergence

```

def visualize_convergence(target, history):
    """Show the arm sliding toward target."""
    fig, ax = plt.subplots(figsize=(10, 8))

    # Workspace
    theta = np.linspace(0, 2*np.pi, 100)
    ax.plot((L1+L2)*np.cos(theta), (L1+L2)*np.sin(theta), 'g--', alpha=0.3)
    ax.plot(abs(L1-L2)*np.cos(theta), abs(L1-L2)*np.sin(theta), 'r--', alpha=0.3)

    # Show iterations
    n = len(history)
    colors = plt.cm.Blues(np.linspace(0.3, 1.0, n))
    for i, q in enumerate(history):
        plot_arm(ax, q, color=colors[i], alpha=0.5)

    # Target
    ax.plot(target[0], target[1], 'r*', markersize=20, label='Target')

    ax.set_xlim(-0.5, 2)
    ax.set_ylim(-0.5, 1.5)
    ax.set_aspect('equal')
    ax.grid(True, alpha=0.3)
    ax.legend()
    ax.set_title(f'Numerical IK: {n} iterations')

    plt.savefig('numerical_ik_convergence.png', dpi=150)
    plt.show()

visualize_convergence(target, history)

```

## 4 Part 3: The Singularity Problem (20 minutes)

### 4.1 3.1 Test Near Singularity

```
# Target near workspace boundary (will cause singularity issues)
target_boundary = [1.45, 0.1] # Near maximum reach
q0 = [0.5, 0.5]

try:
    q_sol, history, success = numerical_ik(target_boundary, q0)
    print(f"Success: {success}")
    print(f"Iterations: {len(history)}")
except Exception as e:
    print(f"Error: {e}")
```

### 4.2 3.2 Exercise 3: Observe Singularity Behavior

Test with different targets:

```
test_targets = [
    ([1.0, 0.5], "Normal - inside workspace"),
    ([1.4, 0.0], "Near boundary"),
    ([1.5, 0.0], "On boundary (singular)"),
    ([1.6, 0.0], "Outside workspace"),
]

for target, description in test_targets:
    print(f"\n{description}: {target}")
    try:
        q_sol, history, success = numerical_ik(target, [0.5, 0.5], max_iter=50)
        J = jacobian_2link(q_sol)
        det_J = np.linalg.det(J)
        print(f"  Success: {success}, Iterations: {len(history)}, det(J)={det_J:.4f}")
    except Exception as e:
        print(f"  Error: {e}")
```

#### Questions:

1. What happens to  $\det(J)$  as we approach the boundary?

*Answer:*

2. Why does the solver fail at the boundary?

*Answer:*

## 5 Part 4: Damped Least Squares (25 minutes)

### 5.1 4.1 Implement DLS Solver

```
def numerical_ik_dls(target, q0, damping=0.05, tol=1e-6, max_iter=100):
    """
    Numerical IK with Damped Least Squares.
    """
    q = np.array(q0, dtype=float)
    history = [q.copy()]

    for i in range(max_iter):
        pos = fk_2link(q)
        error = np.array(target) - pos

        if np.linalg.norm(error) < tol:
            return q, history, True

        J = jacobian_2link(q)

        # Damped Least Squares
        JJT = J @ J.T
        dq = J.T @ np.linalg.solve(JJT + damping**2 * np.eye(2), error)

        q = q + dq
        history.append(q.copy())

    return q, history, False
```

### 5.2 4.2 Exercise 4: Compare Methods

```
target = [1.4, 0.1] # Near boundary
q0 = [0.5, 0.5]

print("Without damping:")
try:
    q1, h1, s1 = numerical_ik(target, q0, max_iter=50)
    print(f" Success: {s1}, Iterations: {len(h1)}")
except:
    print(" Failed!")

print("\nWith damping (=0.05):")
q2, h2, s2 = numerical_ik_dls(target, q0, damping=0.05, max_iter=100)
print(f" Success: {s2}, Iterations: {len(h2)}")

print("\nWith heavy damping (=0.2):")
q3, h3, s3 = numerical_ik_dls(target, q0, damping=0.2, max_iter=200)
print(f" Success: {s3}, Iterations: {len(h3)}")
```

### 5.3 4.3 Exercise 5: Damping Parameter Study

```
damping_values = [0.01, 0.02, 0.05, 0.1, 0.2]
target = [1.3, 0.3]
```

```

q0 = [0.5, 0.5]

results = []
for d in damping_values:
    q, history, success = numerical_ik_dls(target, q0, damping=d, max_iter=200)
    error = np.linalg.norm(fk_2link(q) - target)
    results.append((d, len(history), error, success))

print("Damping | Iterations | Final Error | Success")
print("-" * 50)
for d, iters, err, succ in results:
    print(f" {d:.2f} | {iters:4d} | {err:.2e} | {succ}")

```

Record the best damping value: = \_\_\_\_\_

## 6 Part 5: Initial Guess Effects (15 minutes)

### 6.1 5.1 Exercise 6: Different Solutions

```
target = [0.9, 0.5] # Reachable with both elbow up and down
guesses = [
    [0.8, 0.8],      # Should find elbow up
    [0.2, -0.8],     # Should find elbow down
    [0, 0],          # Neutral
]

fig, ax = plt.subplots(figsize=(10, 8))

for q0 in guesses:
    q, history, success = numerical_ik_dls(target, q0)
    if success:
        plot_arm(ax, q, alpha=0.7)
        print(f"q0={q0} → 1={np.degrees(q[0]):.1f}°, 2={np.degrees(q[1]):.1f}°")

ax.plot(target[0], target[1], 'r*', markersize=20)
ax.set_xlim(-0.5, 1.8)
ax.set_ylim(-1, 1.2)
ax.set_aspect('equal')
ax.grid(True)
ax.set_title('Different Initial Guesses → Different Solutions')
plt.savefig('initial_guess_effect.png', dpi=150)
plt.show()
```

## 7 Part 6: Using roboticstoolbox (15 minutes)

### 7.1 6.1 Compare with Library

```
from roboticstoolbox import DHRobot, RevoluteDH
from spatialmath import SE3

# Create robot
robot = DHRobot([
    RevoluteDH(a=L1),
    RevoluteDH(a=L2)
], name='2-Link')

# Target
target = [1.0, 0.6]
T_target = SE3.Trans(target[0], target[1], 0)

# Solve with library
sol = robot.ikine_LM(T_target, q0=[0.5, 0.5])
print("roboticstoolbox solution:")
print(f"  q = {np.degrees(sol.q)}")
print(f"  Success: {sol.success}")

# Your solution
q_mine, _, _ = numerical_ik_dls(target, [0.5, 0.5])
print("\nYour solution:")
print(f"  q = {np.degrees(q_mine)}")
```

### 7.2 6.2 Exercise 7: FANUC IK

```
# Load or create FANUC model (from Lab 2)
fanuc = DHRobot([
    RevoluteDH(d=0.330, a=0.050, alpha=np.pi/2),
    RevoluteDH(d=0, a=0.330, alpha=0),
    RevoluteDH(d=0, a=0.035, alpha=np.pi/2),
    RevoluteDH(d=0.335, a=0, alpha=-np.pi/2),
    RevoluteDH(d=0, a=0, alpha=np.pi/2),
    RevoluteDH(d=0.080, a=0, alpha=0),
], name='FANUC')

# Target pose
T_target = SE3.Trans(0.4, 0.2, 0.3) * SE3.RPY([0, np.pi, 0])

# Solve
sol = fanuc.ikine_LM(T_target, q0=np.zeros(6))
if sol.success:
    print("FANUC IK solution found!")
    print(f"Joint angles: {np.degrees(sol.q)}")
    fanuc.plot(sol.q, block=False)
```

## 8 Part 7: Challenge - Handle Unreachable Targets (10 minutes)

### 8.1 7.1 Implement Safe IK

```
def is_reachable(target):
    """Check if target is in workspace."""
    r = np.linalg.norm(target)
    return abs(L1 - L2) <= r <= L1 + L2

def safe_numerical_ik(target, q0, damping=0.05, max_iter=100):
    """IK with proper error handling."""
    # Check reachability
    if not is_reachable(target):
        return None, f"Target {target} outside workspace"

    q, history, success = numerical_ik_dls(target, q0, damping, max_iter=max_iter)

    if not success:
        final_error = np.linalg.norm(fk_2link(q) - target)
        return q, f"Did not converge. Final error: {final_error:.4f}"

    return q, None

# Test
test_targets = [
    [1.0, 0.5], # Reachable
    [2.0, 0.0], # Too far
    [0.1, 0.0], # Too close
    [1.45, 0.0], # Boundary
]

for target in test_targets:
    q, msg = safe_numerical_ik(target, [0.5, 0.5])
    if msg:
        print(f"{target}: {msg}")
    else:
        print(f"{target}: Solution found! q={np.degrees(q)}")
```

## 9 Summary

### 9.1 Checklist

- ☐ Jacobian verified against numerical approximation
- ☐ Basic Newton-Raphson IK working
- ☐ Convergence visualization saved
- ☐ Singularity behavior understood
- ☐ Damped Least Squares implemented
- ☐ Optimal damping parameter found
- ☐ Initial guess effects demonstrated
- ☐ Compared with roboticstoolbox
- ☐ Safe IK with error handling complete

### 9.2 Key Takeaways

1. **Jacobian guides the solver** toward the target
2. **Singularities** cause  $\det(J) \rightarrow 0$ , making  $J^{-1}$  unstable
3. **Damped Least Squares** adds stability (trade-off: slower)
4. **Initial guess** determines which solution we find
5. **Always check reachability** before attempting IK

### 9.3 Next Week

Week 6: Trajectory Generation

- Smooth motion profiles
- Cubic and quintic polynomials
- Introduction to FANUC hardware